Test report 10 – CLI daemon

cad-cli (daemon, client, host, config), cad-ipc, cad-engine, cad-session, kernel-smlib · 2026-10-23 · Branch worktree-agent-aaefce8a64cd87ac5 · Commit 75d1f98 · rustc 1.97.0 (2d8144b78 2026-07-07) · 4 cores (Intel(R) Xeon(R) Processor @ 2.10GHz) · measured with `target/release/cad`

cargo test -- workspace	Daemon-Tests	clippy · fmt	`render` cold → 2nd call	Speed-up 2nd call
353 / 353	**12 / 12**	**clean · clean**	**445–948 → 145–209 ms**	**2.8–25.8×**
0 ignored	`crates/cad-cli/tests/daemon.rs`	-D warnings	median, three models	over all 9 series

Empty call (cube)	Memory estimate
32.7 / 30.4 ms	**1.01× (1.00–1.55)**
with / without daemon	heap measurement ÷ held, 48 models

1 Task and result

Report 8 measured 0.4–1.1 s for a single `cad render`, of which the kernel took up to 640 ms; `cad watch` with a warm cache 24 ms. Every CLI call started with an empty cache. Now a background process `cad daemon serve` keeps the engine cache across calls: the first computing command starts it, it ends after 15 min without a request, the cache size is limited, and it is off with `--no-daemon`, `CAD_NO_DAEMON=1` or `[daemon] enabled = false`. Output, exit codes, files and JSON are the same as without the daemon; a second call on the same file takes 34–209 ms instead of 287–1,357 ms.

2 Architecture

```
cad check|eval|inspect|render|export|outline|measure|set …     (client, one process per call)
  1 cad-ipc: connect("daemon") → hello {proto, build_id, pid}
      no daemon     → start cad daemon serve (setsid, stdin /dev/null, stdout/stderr → daemon.log),
                      wait up to 5 s for hello
      other build ID → stop (graceful, SIGKILL after 3 s), start a new one
      error (no folder, foreign socket, Windows) → silently compute locally
  2 run {argv, cwd, stdin?} ──────────►  connection thread ── channel ──► worker thread (256 MiB stack)
     ◄── accepted, wait (1 s tick)       (per connection)                 one Engine<SmlibKernel> for all
projects
     ◄── stdout, stderr, exit, files                                      chdir(cwd); Cli::try_parse_from(argv);
  3 write files (atomic for set),                                         run(&cmd) with a session on the engine
    print stdout/stderr, exit code                                        (host.rs: record files, stdin)
                                       monitor thread: idle 15 min, watchdog 60 s, socket present?
daemon.lock (flock, PID + build) · daemon.sock (0600) · daemon.log  in  $CAD_IPC_DIR |
$XDG_RUNTIME_DIR/controller-cad | /tmp/controller-cad-<uid>
```

- **Same code with and without the daemon.** The CLI commands run unchanged; three places go through `host.rs`: `load()` gets a session via `host::new_session` (in the daemon on its engine, `Session::with_engine`; when the command ends, `host::Pooled` returns the engine), stdin comes from the client, and file writes (`render` PNG, `export`, `--all-parts` folder, `set` atomic) are recorded in the daemon and executed by the client. Output and exit code are built by the same function `finish()` in the daemon as locally. If writing fails on the client, it recomputes locally and thus gives the same error message.
- **Same face tags.** The engine assigns tags in the order in which it first sees node contents; in an engine that has already computed other models, the numbers differ from those in a fresh process (visible in `eval --json`: `nodes[].tag`, `tag_nodes`). The session therefore numbers tags itself in the order of its jobs – exactly the numbers

of a fresh engine – and translates scene and meshes (`render` colors via this mapping). With its own engine the translation is the identity and costs nothing (browser, desktop).
- **Working directory:** before each command the daemon changes to the client's directory (afterwards to `/`); relative paths, messages and `include` behave as locally. One worker thread, commands run one after another (SMLib has global state).
- **Exactly one daemon:** `flock` on `daemon.lock` (new in `cad-ipc`: `LockFile`, file removed at the end, inode check against removal by the predecessor). A process that does not get the lock waits up to 10 s and exits as soon as a daemon responds; several simultaneous clients therefore at most start processes that exit again immediately.
- **Version:** `hello` returns protocol and build ID (version + size and modification time of the executable); on mismatch the client stops the old daemon and starts a new one.
- **Watchdog:** if a command runs longer than 60 s, the daemon reports "watchdog" to the waiting clients and exits (the hanging kernel call cannot be aborted, as with the engine process of the desktop app). The client repeats the command without the daemon and says so on stderr. After 10 s without a sign of life (`wait` arrives every second) the client kills the daemon (SIGKILL) and computes itself.
- **Security:** only the Unix socket from `cad-ipc` (user's folder 0700, socket 0600, peer UID), no network; requests only `hello`, `run`, `status`, `stop`; `run` only for the eight commands (checked with the CLI parser). The daemon reads files as the same user and writes none.

Example

```
$ cad check examples/spec/variabler_radius.scad --verbose
cad: Daemon gestartet (pid 20939, Log $TMP/ipc/daemon.log)
cad: Daemon pid 20939: 626 ms, Cache 1 Treffer / 11 neu, 11 Einträge, 15,4 MiB
examples/spec/variabler_radius.scad: 0 Fehler, 0 Warnungen
$ cad render examples/spec/variabler_radius.scad -o v.png --verbose
cad: Daemon pid 20939: 357 ms, Cache 12 Treffer / 1 neu, 12 Einträge, 15,6 MiB
$TMP/v.png: 1024x768, Ansicht iso, Hülle 160 × 30 × 20 mm
$ cad export examples/spec/lochplatte.scad -o l.step --verbose
cad: Daemon pid 20939: 925 ms, Cache 6 Treffer / 19 neu, 31 Einträge, 22,2 MiB
$TMP/l.step: 2 Körper, 23 Flächen, 49 Kanten, 28669 Bytes, Volumen 2376.6812 mm³
$ cad check examples/spec/lochplatte.scad --json
{"command": "check", "ok": true, "daemon": {"reason": null, "used": true}}
$ cad daemon status
Daemon läuft: pid 20939, Version 0.1.0 (Build 0.1.0+1e0f060.18e128aaa5a04ed5)
Laufzeit 2 s, Leerlauf 0 s (Ende nach 15 min ohne Anfrage), Watchdog 60 s
Anfragen: 4, Projekte 2 von 8 (0 verdrängt)
Cache: 27 Knoten + 4 Netze/abgeleitete, 22,2 MiB von 512,0 MiB, Trefferquote 58 % (42 Treffer, 31 neu
berechnet), 0 verdrängt
Speicher (RSS): 75,1 MiB, Start 35,3 MiB, Cache wird geleert über 803,3 MiB (0× geleert)
Socket $TMP/ipc/daemon.sock, Log $TMP/ipc/daemon.log
```

Log (`daemon.log` in the socket folder):

```
[2026-10-23T12:37:03] cad daemon 20849: Ende (cad daemon stop)
[2026-10-23T12:37:05] cad daemon 20939: Start: Version 0.1.0, Build 0.1.0+1e0f060.18e128aaa5a04ed5, Cache 512
MiB / 16384 Einträge, 8 Projekte, Leerlauf 900 s, Watchdog 60 s, RSS 35,3 MiB
[2026-10-23T12:37:05] cad daemon 20939: `cad check examples/spec/variabler_radius.scad --verbose` → 0 in 626
ms, Cache 1 Treffer / 11 neu, 11 Einträge, 15,4 MiB
[2026-10-23T12:37:06] cad daemon 20939: `cad render examples/spec/variabler_radius.scad -o $TMP/v.png --
verbose` → 0 in 357 ms, Cache 12 Treffer / 1 neu, 12 Einträge, 15,6 MiB
[2026-10-23T12:37:07] cad daemon 20939: `cad export examples/spec/lochplatte.scad -o $TMP/l.step --verbose` → 0
in 925 ms, Cache 6 Treffer / 19 neu, 31 Einträge, 22,2 MiB
[2026-10-23T12:37:07] cad daemon 20939: `cad check examples/spec/lochplatte.scad --json` → 0 in 1 ms, Cache 23
Treffer / 0 neu, 31 Einträge, 22,2 MiB
```

3 Limits and configuration

Setting	Default	File `[daemon]`	Environment	`cad daemon serve`

Daemon on/off	on	`enabled = false` (or `daemon = false` at top level)	`CAD_NO_DAEMON=1`	– (per call `--no-daemon`)
Cache (node results + meshes)	512 MiB	`cache_mb`	`CAD_DAEMON_CACHE_MB`	`--cache-mb`
Entries	16384	`cache_entries`	`CAD_DAEMON_CACHE_ENTRIES`	–
Projects in the cache	8	`max_projects`	`CAD_DAEMON_MAX_PROJECTS`	`--max-projects`
Idle time until exit	15 min	`idle_minutes`, `idle_secs`	`CAD_DAEMON_IDLE_MINUTES`, `_IDLE_SECS`	`--idle-minutes`, `--idle-secs`
Watchdog	60 s	`watchdog_secs`	`CAD_DAEMON_WATCHDOG_SECS`	`--watchdog-secs`
RSS limit	start + 1.5 × cache	`rss_factor`	`CAD_DAEMON_RSS_FACTOR`	–

File: `$CAD_CONFIG`, else `$XDG_CONFIG_HOME/controller-cad/config.toml`, else `~/.config/controller-cad/config.toml`; precedence arguments > environment > file > default; invalid values are ignored (log, with `--verbose` on stderr).

- **Cache:** one LRU of the engine for node results and meshes together (before: 4096 nodes without size, meshes 256 MiB separately), with a byte limit and an entry limit; the newest entry stays even if it alone exceeds the limit. One engine for all projects; the keys are content-based.
- **Projects:** per main file (absolute path; stdin per working directory) the daemon remembers the keys its commands touched. With the 9th project, the least recently used one loses its entries unless another held project uses them.
- **RSS:** after each request from `/proc/self/statm`; above start RSS + 1.5 × cache limit the cache is cleared and free memory returned with `malloc_trim`. The start RSS (about 35–40 MiB: executable, SMLib, Tauri libraries) does not count against the limit, otherwise a small limit would always trigger.
- **Lifetime:** exit after 15 min without `run` (`status` does not count), after `cad daemon stop` (waits for running commands), when the socket is removed or replaced (e.g. test folder deleted), after the watchdog; socket and lock file are removed.

Memory estimate

Kernel results are handles in SMLib; only the C++ heap knows their size. Measured with `crates/cad-cli/examples/cache_size.rs`: evaluate each of the 48 example models with a fresh engine, then drop the engine and count the freed heap bytes (glibc `mallinfo2`). The breps hold 10–297 KiB per face (median 26 KiB), free-form surfaces and variable fillets the most. The first estimate from the topology (4 KiB + 3 KiB/face + 1 KiB/edge) was too low by a factor of 6. Now: `Kernel::shape_bytes` for SMLib 8 KiB + 32 KiB per face + 1 KiB per edge (sum over all examples 1.10×, per model 0.13–4.54×) and in the daemon additionally a measurement per node: heap after minus before the evaluation of the node (`Engine::set_heap_probe`), which captures the new breps of the node including SMLib intermediate data. The estimate per model is thus 1.00–1.55× of what is actually held (median 1.01), never below.

Model	Breps	Faces	Edges	held (MiB)	topology estimate (÷ held)	daemon: heap measurement (÷ held)
spec/freiformdeckel.scad	5	91	172	18.24	4.13 (0.23)	18.38 (1.01)
spec/variabler_radius.scad	11	169	336	14.91	5.71 (0.38)	15.25 (1.02)
spec/knauf_mirror.scad	16	48	67	13.92	1.81 (0.13)	16.39 (1.18)
spec/dose_shell.scad	6	218	404	12.13	10.80 (0.89)	12.48 (1.03)
spec/kabelfuehrung.scad	8	104	246	9.56	3.56 (0.37)	9.58 (1.00)
spec/loft_sweep.scad	6	45	87	8.78	1.54 (0.18)	8.79 (1.00)
openscad-compat/20_box_with_lid.scad	34	366	732	8.52	13.09 (1.54)	8.51 (1.00)

spec/griffschale.scad	16	201	462	6.45	8.53 (1.32)	6.96 (1.08)
all 48 models	**559**	**4989**	**9887**	**180.1**	**198.6 (1.10)**	**190.7 (1.06)**

The eight largest models; "held" = freed when the engine is dropped. Meshes count 4 bytes per array element (exact).

4 Measurement

5 repetitions per row, each with a stopped daemon and a fresh copy of the file; median (bold), next to it minimum–maximum, wall-clock time of the `cad` process in ms (2026-10-23 12:37, 4 cores). "Cold without daemon" = behavior up to report 9. Small changes:

- `spec/variabler_radius.scad`: pocket `cube([30, 14, 10])` → `[30, 15, 10]` (one body of three rebuilt, with variable fillet)
- `projekt-gehaeuse/gehaeuse.scad`: lid moved by 1 mm (`tiefe + abstand + 1`; only the root rebuilt)
- `spec/lochplatte.scad`: pin `d = 3.5` → `3.6` (cylinder and root rebuilt)

Model	Command	cold without daemon	1st call with daemon (incl. start)	2nd call	2nd call after small change	cold ÷ 2nd
spec/variabler_radius.scad	`check`	**625** 550–768	**695** 567–766	**36** 32–42	**162** 154–246	17.5×
spec/variabler_radius.scad	`render -o x.png`	**948** 890–1,163	**1,096** 978–1,326	**145** 139–205	**507** 493–619	6.5×
spec/variabler_radius.scad	`export -o x.step`	**1,357** 1,267–1,754	**1,313** 1,275–1,575	**63** 57–72	**1,003** 797–1,094	21.4×
projekt-gehaeuse/gehaeuse.scad	`check`	**287** 267–329	**331** 306–497	**35** 31–42	**36** 33–46	8.3×
projekt-gehaeuse/gehaeuse.scad	`render -o x.png`	**445** 427–465	**516** 457–662	**156** 149–161	**188** 183–229	2.8×
projekt-gehaeuse/gehaeuse.scad	`export -o x.step`	**409** 391–496	**425** 424–652	**34** 32–41	**153** 128–167	12.1×
spec/lochplatte.scad	`check`	**589** 519–759	**698** 586–882	**37** 32–43	**97** 73–154	15.7×
spec/lochplatte.scad	`render -o x.png`	**933** 874–1,090	**962** 851–1,031	**209** 190–224	**331** 327–392	4.5×
spec/lochplatte.scad	`export -o x.step`	**965** 811–1,338	**977** 858–1,530	**37** 34–44	**550** 403–666	25.8×

- The 1st call with the daemon costs about as much as cold without the daemon: median 44 ms more (process start of the daemon, waiting for `hello`, one additional transfer), per series -44 to +149 ms – within the spread of the cold time.
- The 2nd call computes nothing in the kernel: `check` needs only parser, evaluation and the round trip over the socket; `render` additionally rasterization and PNG (the mesh comes from the cache), `export -o x.step` the STEP writer.
- After a small change the kernel computes only the affected nodes (column "after small change"). For `variabler_radius` the one new variable fillet dominates; for the STEP export the B-rep of the changed root is added, which is read anew from SMLib and written.
- An empty call (`check` of `cube(1)`) costs 32.7 ms with the daemon, 30.4 ms without: process start of the client and socket instead of SMLib initialization in the process.

5 Tests

- `cargo test --workspace`: 353 passed, 0 failed, 0 ignored; `cargo clippy --workspace --all-targets -- -D warnings`: no findings; `cargo fmt --all --check`: no findings.
- Each test has its own socket folder (`CAD_IPC_DIR` in a temporary directory) and stops its daemon (guard with `cad daemon stop`); if the test deletes the folder, the daemon exits by itself at most 0.5 s later (socket missing). The other CLI tests (`cli.rs`, `agent.rs`, `projects.rs`) run with `CAD_NO_DAEMON=1`.
- CI: `scripts/ci/rust.sh test` checks after `cargo test` that no `cad daemon serve` is left over (in CI, error otherwise); `scripts/ci/compat.sh compare|step` now run through the daemon (own socket folder, then `status` and `stop`), so the OpenSCAD comparison and the STEP check with OpenCASCADE also check the daemon's output; the Tauri E2E tests call `cad` with `CAD_NO_DAEMON=1`.

Test (`crates/cad-cli/tests/daemon.rs`)	Checks	Result
`changed_files_are_never_stale`	main file, included file, return to an old state, `set` via the client, syntax error and fix – every result matches the current content	ok
`autostart_then_cache_hits_and_faster`	first call starts the daemon (`daemon.used`), second call: all nodes from the cache, 0 kernel calls, same output, hit counter increases, measurably faster	ok
`fallback_when_socket_is_broken`	socket folder is a file; foreign process on the socket that closes every connection; stale socket without process (gets replaced)	ok
`idle_end_removes_socket_and_lock`	idle 1 s: daemon exits, socket and lock file removed, log „ohne Anfrage“ ("without request")	ok
`parallel_starts_give_one_daemon`	six simultaneous calls without daemon: all use the daemon, exactly one `cad daemon serve` process remains	ok
`relative_paths_and_stdin`	relative path with another working directory (same output), source via stdin (`-`)	ok
`status_and_stop`	`daemon status` (text, JSON fields), `daemon stop` (PID, socket removed), second `stop`: „kein Daemon“ ("no daemon")	ok
`switched_off_by_flag_env_and_config`	`--no-daemon` (before and after the command), `CAD_NO_DAEMON=1`, `[daemon] enabled = false`: no daemon, reason in `daemon.reason`, text silent, `--verbose` names the reason	ok
`version_change_replaces_daemon`	build ID a → b: client stops the old daemon and starts a new one; `status` of a third build reports the mismatch	ok
`watchdog_ends_daemon_and_command_runs_locally`	watchdog 1 s, artificially hanging command (8 s): daemon exits, command runs without daemon with the same output, message on stderr, next call starts a new daemon	ok
`cache_limit_and_project_limit`	limit 1 MiB and 2 projects with 7 models: evictions, size below the limit (or only the newest entry), 5 projects evicted; 512 MiB: nothing evicted	ok
`output_identical_with_and_without_daemon`	daemon first filled with three other models; then per command locally, 1st and 2nd call with daemon: check (ok, syntax error, kernel error), eval, eval --module, inspect, outline, measure (distance/volume, collision/clearance), set --dry-run, export with unknown part (exit 2) – text and JSON; render (contact sheet, section, X-ray) PNG byte by byte, STL, 3MF, STEP (without timestamp), --all-parts into a folder	ok
`lock_file_is_exclusive_and_removed`	`cad-ipc`: lock file exclusive (also within the same process), 0600, removed at the end	ok
`config::tests::file_values_and_errors`	configuration file: values, unknown keys, `daemon = false`, `enabled = false`, broken TOML	ok
`cache::tests::evicts_least_recently_used`	LRU: entry limit, `retain`, eviction counter	ok

`cache::tests::entry_limit_and_retain`	LRU: entry limit, `retain`, eviction counter	**ok**

In the test `autostart_then_cache_hits_and_faster` (debug build, `eval --json` of `variabler_radius`): 1st call 680 ms, 2nd call 49 ms.

6 Known limitations

- Unix only (Linux, macOS; peer UID check only on Linux, otherwise the 0700 folder protects). On Windows every command computes by itself, silently.
- One worker thread: simultaneous CLI calls wait for each other (without the daemon they would run in parallel in separate processes); `wait` events keep the clients alive.
- The watchdog also hits a legitimately long command (> 60 s): the daemon exits, the command runs a second time without the daemon. A crash in the kernel (signal) also ends the daemon; the local repetition can crash the same way. The watchdog is tested with an artificial delay (`CAD_DAEMON_DEBUG_HANG_MS`), not with a real hanging SMLib call.
- Memory measurement per node only with glibc; allocations of other threads (connections) are counted too. Without glibc the topology estimate applies (per model factor 0.1–4.5). The RSS is checked after each request, not during an evaluation.
- Differences from the output without the daemon: the additional JSON field `daemon`; counters and timings (`stats.cache_hits`, `kernel_calls`, `cached`, `*_ms`, `timings`, the „Statistik“ line of `eval`) show what actually happened. On a panic, message and exit code are the same, but the runtime's panic message goes to the log instead of stderr.
- Build ID from size and modification time of the executable: if two `cad` binaries alternate (debug and release), they replace each other's daemon (cache lost). An old daemon that is still computing is killed after 3 s; its client then computes itself.
- The engine's tag table keeps growing (one entry per node content seen); after 2^{20} tags the daemon replaces the engine together with its cache.
- `cad watch` does not use the daemon (own cache in the process); `cad set` checks the change with a second, kernel-less session without cache (as before).

7 Reproduction

```
export SMLIB_PREBUILT=$PWD/smlib/build-native
cargo build --release -p cad-cli --bin cad --example cache_size
python3 tools/report/report10-measure.py --cad target/release/cad --repeat 5
target/release/examples/cache_size examples/spec/*.scad examples/projekt-gehaeuse/gehaeuse.scad \
    examples/openscad-compat/*.scad > reports/tmp/report10/cache_size.jsonl
cargo test --workspace --no-fail-fast > reports/tmp/report10/cargo.txt 2>&1
cargo test -p cad-cli --test daemon -- --nocapture --test-threads 1 > reports/tmp/report10/daemon-tests.txt
2>&1
cargo clippy --workspace --all-targets -- -D warnings > reports/tmp/report10/clippy.txt 2>&1
cargo fmt --all --check > reports/tmp/report10/fmt.txt
node tools/report/report10.mjs
node tools/report/print-pdf.mjs docs/reports/10-cli-daemon/report.html docs/reports/10-cli-daemon/report.pdf
"Test report 10 – CLI daemon"
```